SISMID Geospatial Gentrification Lab

Lance A. Waller

2026-07-14

TADA Gentrification Index Merging Census Data

Goal of the lab: We want to use R to map census tract level data, read in an Excel file (saved as a "comma separated value" (.csv) file of additional demographic data), merge the new data to the map data, calculate the gentrication index from Corrigan et al. (2021, Applied Geography) and map the index.

To start we use the online text Analyzing US Census Data: Methods, Maps, and Models in R by Kyle Walker... online markdown at https://walker-data.com/census-r/index.html. Specifically, we'll use Chapter 5 (Census geographic data and applications in R https://walker-data.com/census-r/census-geographic-data-and-applications-in-r.html) and Chapter 6 (Mapping Census data in R) https://walker-data.com/census-r/mapping-census-data-with-r.html

Next we learn to read in TIGER (Topographically Integrated GEographic Reference) files... the map boundary data from the US Census.

These data are in the tigris R package. We will also need tidycensus and tidyverse

```
library(tigris)
```

```
## To enable caching of data, set `options(tigris_use_cache = TRUE)`
## in your R script or .Rprofile.
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.1.5
## v ggplot2   3.5.2     v stringr   1.5.1
## v lubridate 1.9.4     v tibble    3.3.0
## v purrr     1.0.4     v tidyr     1.3.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Reading in and mapping census tracts

```r
library(tidycensus)
# Add your tidycensus API key from our earlier lab here
census_api_key("7efec8b05c7c64f71a8765a85f0f7d1e4b4e0599")
```

```
## To install your API key for use in future sessions, run this function with `install = TRUE`.
```

```r
options(tigris_use_cache = TRUE)

dc_income <- get_acs(
  geography = "tract",
  variables = "B19013_001",
  state = "DC",
  year = 2020,
  geometry = TRUE
)
```

```
## Getting data from the 2016-2020 5-year ACS
```

```r
dc_income
```

```
## Simple feature collection with 206 features and 5 fields
## Geometry type: POLYGON
## Dimension:     XY
## Bounding box:  xmin: -77.11976 ymin: 38.79165 xmax: -76.9094 ymax: 38.99511
## Geodetic CRS:  NAD83
## First 10 features:
##          GEOID                                                          NAME
## 1  11001001002 Census Tract 10.02, District of Columbia, District of Columbia
## 2  11001002801 Census Tract 28.01, District of Columbia, District of Columbia
## 3  11001003100    Census Tract 31, District of Columbia, District of Columbia
## 4  11001004001 Census Tract 40.01, District of Columbia, District of Columbia
## 5  11001010500   Census Tract 105, District of Columbia, District of Columbia
## 6  11001004600    Census Tract 46, District of Columbia, District of Columbia
## 7  11001008803 Census Tract 88.03, District of Columbia, District of Columbia
## 8  11001009507 Census Tract 95.07, District of Columbia, District of Columbia
## 9  11001009509 Census Tract 95.09, District of Columbia, District of Columbia
## 10 11001005303 Census Tract 53.03, District of Columbia, District of Columbia
##      variable estimate   moe                       geometry
## 1  B19013_001   140772 20766 POLYGON ((-77.08563 38.9382...
## 2  B19013_001    62323 22218 POLYGON ((-77.03645 38.9349...
## 3  B19013_001   133408 18433 POLYGON ((-77.02826 38.9318...
## 4  B19013_001   153650  5886 POLYGON ((-77.05018 38.9212...
## 5  B19013_001    92083 20106 POLYGON ((-77.01756 38.8850...
## 6  B19013_001   129896 18312 POLYGON ((-77.01811 38.9146...
## 7  B19013_001    60819 22048 POLYGON ((-77.00173 38.9099...
## 8  B19013_001    72880 18411 POLYGON ((-77.00229 38.9567...
## 9  B19013_001    86042 25212 POLYGON ((-77.00201 38.9510...
## 10 B19013_001    97181  9127 POLYGON ((-77.04298 38.9101...
```

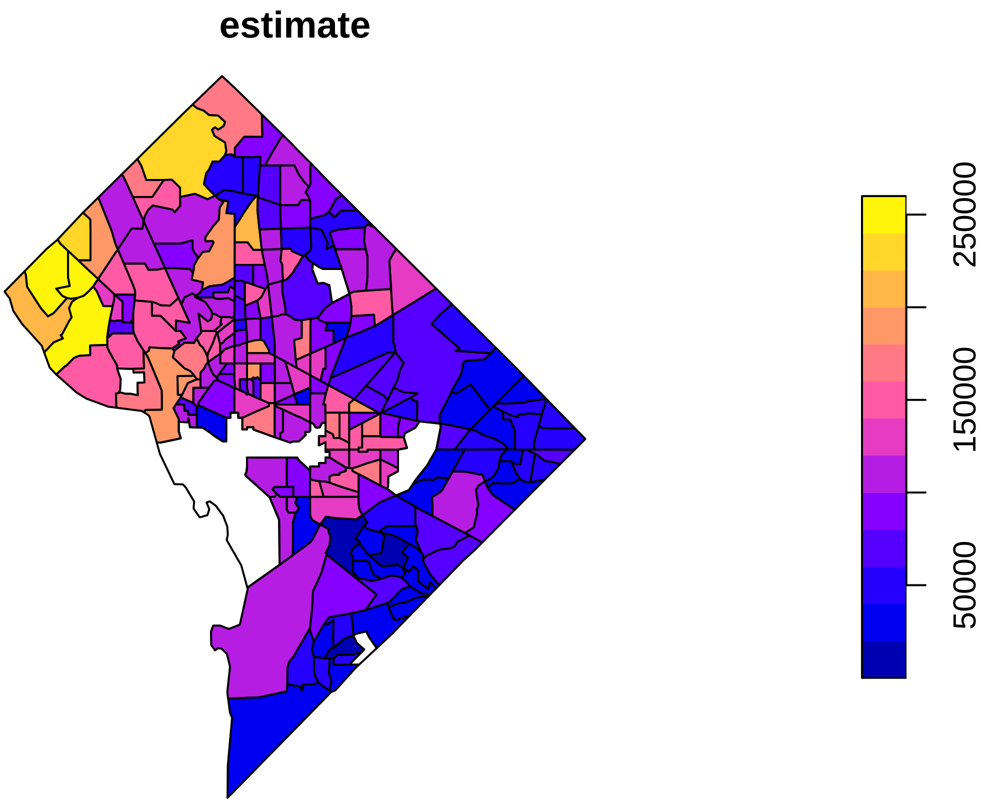
From the book: "Median household income data are found in the estimate column with associated margins of error in the moe column, along with a variable ID, GEOID, and Census tract name. However, there are some notable differences. The geometry column contains polygon feature geometry for each Census tract, allowing for a linking of the estimates and margins of error with their corresponding locations in Washington, DC. Beyond that, the object is associated with coordinate system information - using the NAD 1983 geographic coordinate system in which Census geographic datasets are stored by default.”

Lance’s note: GEOID is the FIPS code for the Census tracts in Washington DC. . . we will need to use this identifier to merge data in our work below.

Mapping

From the book: “Such geographic information can be difficult to understand without visualization. As the returned object is a simple features object, both geometry and attributes can be visualized with plot(). Key here is specifying the name of the column to be plotted inside of brackets, which in this case is”estimate”.”

```
plot(dc_income["estimate"])
```



Cool! We can read in tract data from tigris and map it with the plot function!

Let’s see if we can read in tracts from Georgia. We’ll change the state to “GA” and rename our income variable.

```
ga_income <- get_acs(
  geography = "tract",
  variables = "B19013_001",
  state = "GA",
  year = 2016,
  geometry = TRUE
)
```

```
## Getting data from the 2012-2016 5-year ACS
```

```
ga_income
```

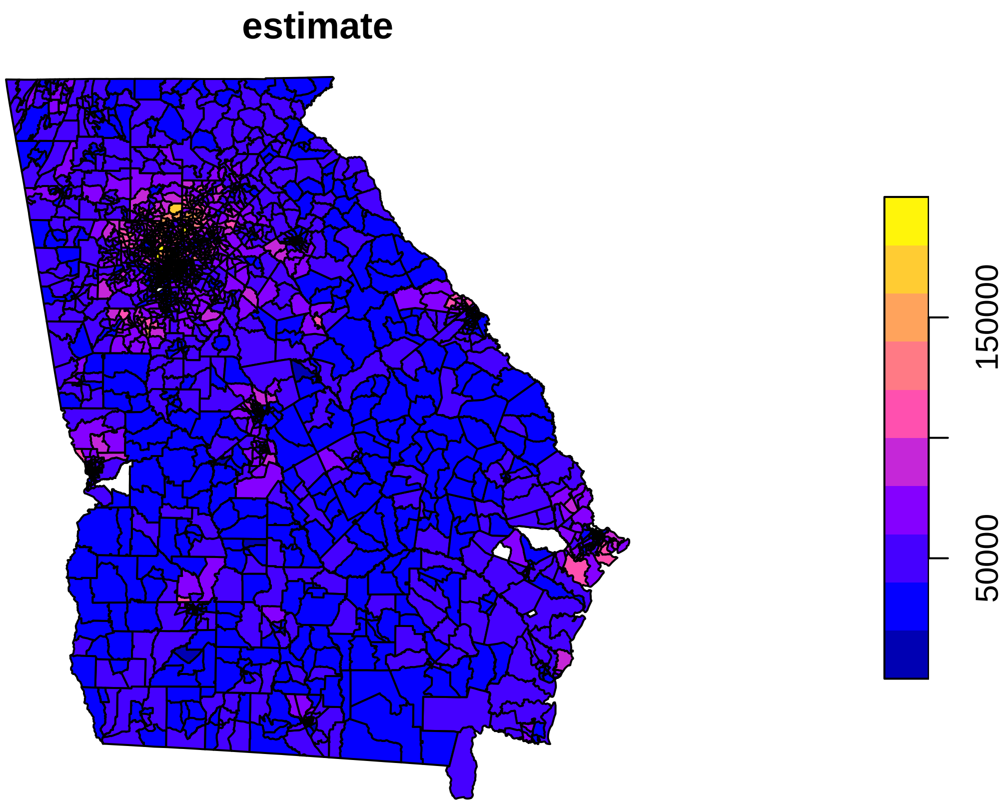
```
## Simple feature collection with 1969 features and 5 fields (with 3 geometries empty)
## Geometry type: MULTIPOLYGON
```

```
## Dimension:     XY
## Bounding box:  xmin: -85.60516 ymin: 30.35785 xmax: -80.83973 ymax: 35.00066
## Geodetic CRS:  NAD83
## First 10 features:
##          GEOID                                          NAME   variable
## 1  13089023501   Census Tract 235.01, DeKalb County, Georgia B19013_001
## 2  13095000502  Census Tract 5.02, Dougherty County, Georgia B19013_001
## 3  13095001000    Census Tract 10, Dougherty County, Georgia B19013_001
## 4  13097080304  Census Tract 803.04, Douglas County, Georgia B19013_001
## 5  13097080508  Census Tract 805.08, Douglas County, Georgia B19013_001
## 6  13101880100     Census Tract 8801, Echols County, Georgia B19013_001
## 7  13113140101 Census Tract 1401.01, Fayette County, Georgia B19013_001
## 8  13115001100        Census Tract 11, Floyd County, Georgia B19013_001
## 9  13117130204 Census Tract 1302.04, Forsyth County, Georgia B19013_001
## 10 13117130602 Census Tract 1306.02, Forsyth County, Georgia B19013_001
##    estimate   moe                       geometry
## 1     31649 10000 MULTIPOLYGON (((-84.28807 3...
## 2     66389 15116 MULTIPOLYGON (((-84.22294 3...
## 3     23770  6059 MULTIPOLYGON (((-84.22209 3...
## 4     31841  9014 MULTIPOLYGON (((-84.77867 3...
## 5     54555  7700 MULTIPOLYGON (((-84.77077 3...
## 6     29839  8640 MULTIPOLYGON (((-83.05618 3...
## 7     71522  6421 MULTIPOLYGON (((-84.55866 3...
## 8     23938  4690 MULTIPOLYGON (((-85.20798 3...
## 9     61447  8983 MULTIPOLYGON (((-84.11487 3...
## 10    88654 13204 MULTIPOLYGON (((-84.25877 3...
```

Note: The first 2 digits of the GEOID (FIPS Code) are now "13" which is the FIPS code for the state of GA.

Let's map it.

```r
plot(ga_income["estimate"])
```



We can add a county list to limit the map to those counties. For the Corrigan paper we want Cobb, Fulton, DeKalb, Gwinnett, and Clayton. We'll call this Atlanta metro.

```
metro_atlanta_income <- get_acs(
  geography = "tract",
  variables = "B19013_001",
  state = "GA",
  county = c("Fulton","DeKalb","Cobb","Gwinnett","Clayton"),
  year = 2016,
  geometry = TRUE
)
```

```
## Getting data from the 2012-2016 5-year ACS
```

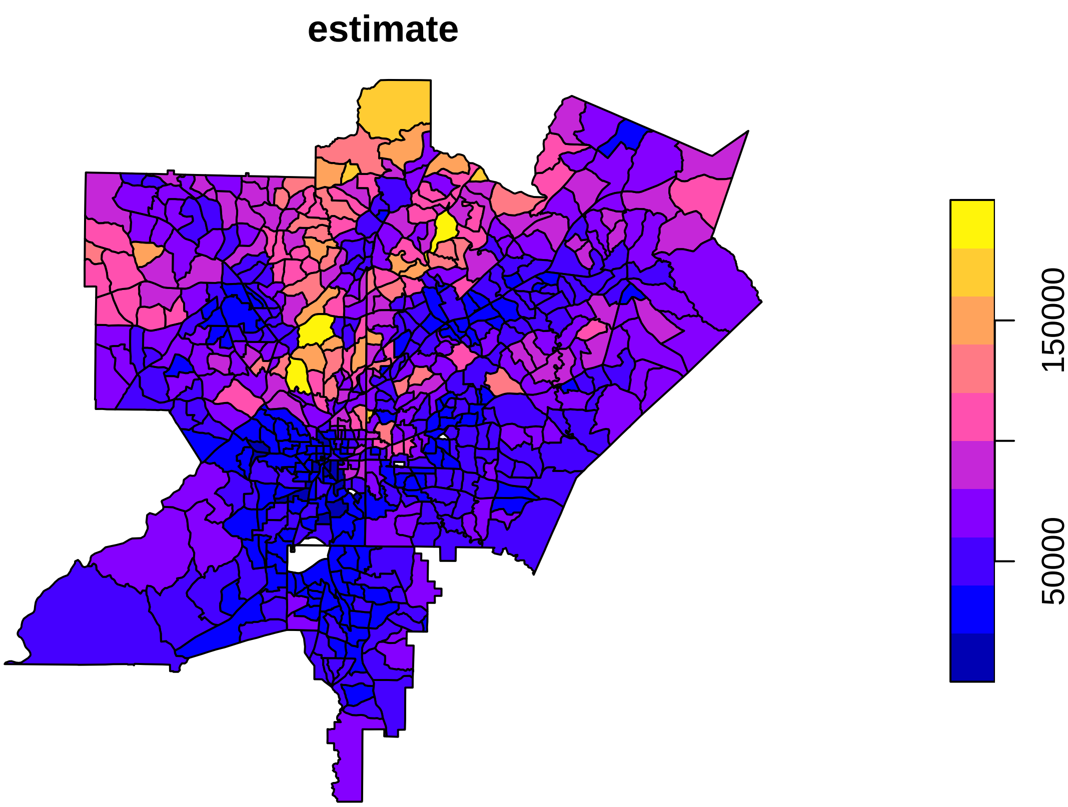
```
metro_atlanta_income
```

```
## Simple feature collection with 632 features and 5 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -84.85071 ymin: 33.35246 xmax: -83.7991 ymax: 34.18629
## Geodetic CRS:  NAD83
## First 10 features:
##          GEOID                                         NAME   variable estimate
## 1  13089023501  Census Tract 235.01, DeKalb County, Georgia B19013_001    31649
## 2  13121000200       Census Tract 2, Fulton County, Georgia B19013_001   106045
## 3  13121001500      Census Tract 15, Fulton County, Georgia B19013_001    60356
## 4  13121003100      Census Tract 31, Fulton County, Georgia B19013_001    43170
## 5  13063040203 Census Tract 402.03, Clayton County, Georgia B19013_001    32866
## 6  13063040512 Census Tract 405.12, Clayton County, Georgia B19013_001    37578
## 7  13063040615 Census Tract 406.15, Clayton County, Georgia B19013_001    65601
```

```
## 8  13067030224    Census Tract 302.24, Cobb County, Georgia B19013_001    68097
## 9  13067030233    Census Tract 302.33, Cobb County, Georgia B19013_001   101856
## 10 13067030322    Census Tract 303.22, Cobb County, Georgia B19013_001    89286
##      moe                       geometry
## 1  10000 MULTIPOLYGON (((-84.28807 3...
## 2  17937 MULTIPOLYGON (((-84.37421 3...
## 3  10036 MULTIPOLYGON (((-84.35987 3...
## 4   7876 MULTIPOLYGON (((-84.35824 3...
## 5   7081 MULTIPOLYGON (((-84.45853 3...
## 6   7205 MULTIPOLYGON (((-84.44869 3...
## 7   3467 MULTIPOLYGON (((-84.32657 3...
## 8   7966 MULTIPOLYGON (((-84.65095 3...
## 9  13081 MULTIPOLYGON (((-84.62971 3...
## 10 13908 MULTIPOLYGON (((-84.46282 3...
```

Map it

```r
plot(metro_atlanta_income["estimate"])
```



Great!. . . We do notice some "holes" which are census tracts for which there are no ACS data. We will ignore these for now.

Corrigan et al.'s (2021) gentrification score is defined on page 2 of their paper. "A gentrification score was calculated by summing the standarized percent change between time 1 and time 2 for all indicators."

The seven indicators are:

- Percent non-Hispanic white residents,
- Percent of residents aged 25-34 years old,
- Percent of residents that were *not* in poverty,

- Percent of residents working in professional occupations,
- Median gross rent,
- Median house value,
- Median household income in the past 12 months.

Sabriya Linton shared the tables of values for census tracts from 2016 and 2010 in two .csv files.

We will:

- Read in each csv
- Calculate the standardized difference for each tract.
- Merge to the tract file we created above by matching the variable 'tract' in the .csv files to the variable 'GEOID' in the mapped data.
- Map the results

Reading in a csv file

Note: Make sure you save the two .csv files in the same directory as your .Rmd code!

```
gentrification_vars_2010 = read.csv("gentrification_vars_2010.csv")
gentrification_vars_2016 = read.csv("gentrification_vars_2016.csv")
```

First few rows of 2010 data

```
head(gentrification_vars_2010)
```

```
##   X       tract age_25to34_10_pct edu_bs_10_pct owns_10_pct Hhval_med_10
## 1 1 13013180205              14.4          10.0       14.61       131600
## 2 2 13015960802              13.5           8.5       11.55        78800
## 3 3 13045910501              11.9           6.1        7.71       123400
## 4 4 13045910502              12.2           7.4        6.07        96700
## 5 5 13045910600              16.4          18.7       10.77       151800
## 6 6 13063040202              20.6          10.3        9.84       112700
##   inc_Hhmed_10 HH_nonfam_10_pct pov_fams_10_pct prof_ind_10_pct
## 1        38077            37.62            19.6            1.43
## 2        38929            30.38            18.0           13.91
## 3        35810            37.59            15.0            9.21
## 4        17649            49.11            33.5            4.11
## 5        47016            35.55            12.7           10.81
## 6        30946            31.45            20.2            6.80
##   rent_medgross_10 white_nonlat_10_pct
## 1              728                69.7
## 2              728                83.3
## 3              771                51.6
## 4              717                24.8
## 5              753                65.1
## 6              885                 2.3
```

First few rows of 2016 data

```
head(gentrification_vars_2016)
```

```
##   X       tract age_25to34_pct edu_BS_pct owns_pct Hhval_med inc_Hhmed
## 1 1 13013180205           13.6       10.5    42.05    106100     40139
## 2 2 13015960802           12.9        5.6    63.56     60500     33393
```

```
## 3 3 13045910501           13.7         8.6    30.12     88100     35267
## 4 4 13045910502           17.7         2.1     8.89     83300     24367
## 5 5 13045910600           14.6        17.2    69.24    119400     53269
## 6 6 13063040202           18.2         7.9    33.30     82500     28105
##   HH_nonfam_pct pov_fams_pct prof_ind_pct rent_medgross white_nonlat_pct
## 1         39.31         17.7        11.59           950             62.5
## 2         35.29         21.7        14.73           904             83.8
## 3         45.93         14.7         7.99           752             43.8
## 4         48.70         32.5        11.88           645             19.9
## 5         23.95         19.2         7.52           787             65.7
## 6         33.62         30.0        10.59           886              4.9
```

Calculating the standardized differences

To get the standardized difference, we take the difference between the 2016 and 2010 values *for each tract*, subtract the mean difference across tracts, and divide by the standard deviation of differences. For each of the variables, we will need to calculate:

difference = (value in 2016) - (value in 2010) std.diff = (difference - mean(difference))/sd(difference)

NOTE: The data contain some missing values (NAs)…the mean() and sd() functions won't work with NAs so we'll need to ask R to calculate the means and standard deviations without the NAs…one way to do this is set na.rm = TRUE in the calls to the mean() and sd()

```r
# calculate differences
diff.NHWhite = gentrification_vars_2016$white_nonlat_pct - gentrification_vars_2010$white_nonlat_10_pct

diff.2534 = gentrification_vars_2016$age_25to34_pct - gentrification_vars_2010$age_25to34_10_pct

# doing 1-pov to make it percent not in poverty
diff.notpov = (1-gentrification_vars_2016$pov_fams_pct) - (1-gentrification_vars_2010$pov_fams_10_pct)

diff.prof = gentrification_vars_2016$prof_ind_pct - gentrification_vars_2010$prof_ind_10_pct

diff.medrent = gentrification_vars_2016$rent_medgross - gentrification_vars_2010$rent_medgross_10

diff.houseval = gentrification_vars_2016$Hhval_med - gentrification_vars_2010$Hhval_med_10

diff.income = gentrification_vars_2016$inc_Hhmed - gentrification_vars_2010$inc_Hhmed_10
```

Standardize the differences

```r
std.diffNHWhite = (diff.NHWhite - mean(diff.NHWhite,na.rm=TRUE))/sd(diff.NHWhite,na.rm=TRUE)

head(std.diffNHWhite)
```

```
## [1] -0.6566358  0.3754466 -0.7370578 -0.3483515  0.3888503  0.6569236
```

```r
std.diff.2534 = (diff.2534 - mean(diff.2534,na.rm=TRUE))/sd(diff.2534,na.rm=TRUE)
std.diff.notpov = (diff.notpov - mean(diff.notpov,na.rm=TRUE))/sd(diff.notpov,na.rm=TRUE)
std.diff.prof = (diff.prof - mean(diff.prof,na.rm=TRUE))/sd(diff.prof,na.rm=TRUE)
std.diff.medrent = (diff.medrent - mean(diff.medrent,na.rm=TRUE))/sd(diff.medrent,na.rm=TRUE)
std.diff.houseval = (diff.houseval - mean(diff.houseval,na.rm=TRUE))/sd(diff.houseval,na.rm=TRUE)
std.diff.income = (diff.income - mean(diff.income,na.rm=TRUE))/sd(diff.income,na.rm=TRUE)
```

Get scores

```
gent.score = std.diffNHWhite + std.diff.2534 + std.diff.notpov + std.diff.prof + std.diff.medrent + std

head(gent.score)
```

```
## [1]  1.7886630 -0.1311656 -1.6230379  2.1309952 -1.4455037 -1.3332301
```

```
range(gent.score,na.rm=TRUE)
```

```
## [1] -8.493753 14.279389
```

Make a histograph of scores

```
hist(gent.score,10)
```

Check the length of the 'tract' (FIPS code) and the gentrification values.

```
length(gentrification_vars_2010$tract)
```

```
## [1] 655
```

```
length(gent.score)
```

```
## [1] 655
```

Check the tract ids are the same between the 2010 and 2016 data (is the max difference zero?)

```
max(gentrification_vars_2016$tract - gentrification_vars_2010$tract)
```

```
## [1] 0
```

Great, we're ready to go.

Mapping the Gentrification Scores

How many tracts in the mapped data?

```r
length(metro_atlanta_income$GEOID)
```

```
## [1] 632
```

Hmmmm. 632 features. We have 655. Let's match on GEOID and tract.

This involves:

- making a data frame with the 'tract' variable from gentrification_vars_2016, and our new gent.score.
- renaming "tract" (which is variable V1 in our data frame since we didn't name it!)
- Change our new GEOID (which turns out be stored as a number) to a character variable (found this out by trial and error)
- print out the first few rows with 'head'
- merge the data using a 'left_join' command…this will put the variable (gent.score) from the my.gent.data frame into the metro_atlanta_income data we can map from, matched by "GEOID" (both metro_atlanta_income and my.gent.data have GEOID and both are characters so we can match them up.)

```r
my.gent.data = data.frame(cbind(gentrification_vars_2016$tract,gent.score)) %>%
  rename("GEOID" = "V1", "gent_score" = "gent.score") %>%
  mutate(GEOID = as.character(GEOID))

head(my.gent.data)
```

```
##         GEOID gent_score
## 1 13013180205  1.7886630
## 2 13015960802 -0.1311656
## 3 13045910501 -1.6230379
## 4 13045910502  2.1309952
## 5 13045910600 -1.4455037
## 6 13063040202 -1.3332301
```

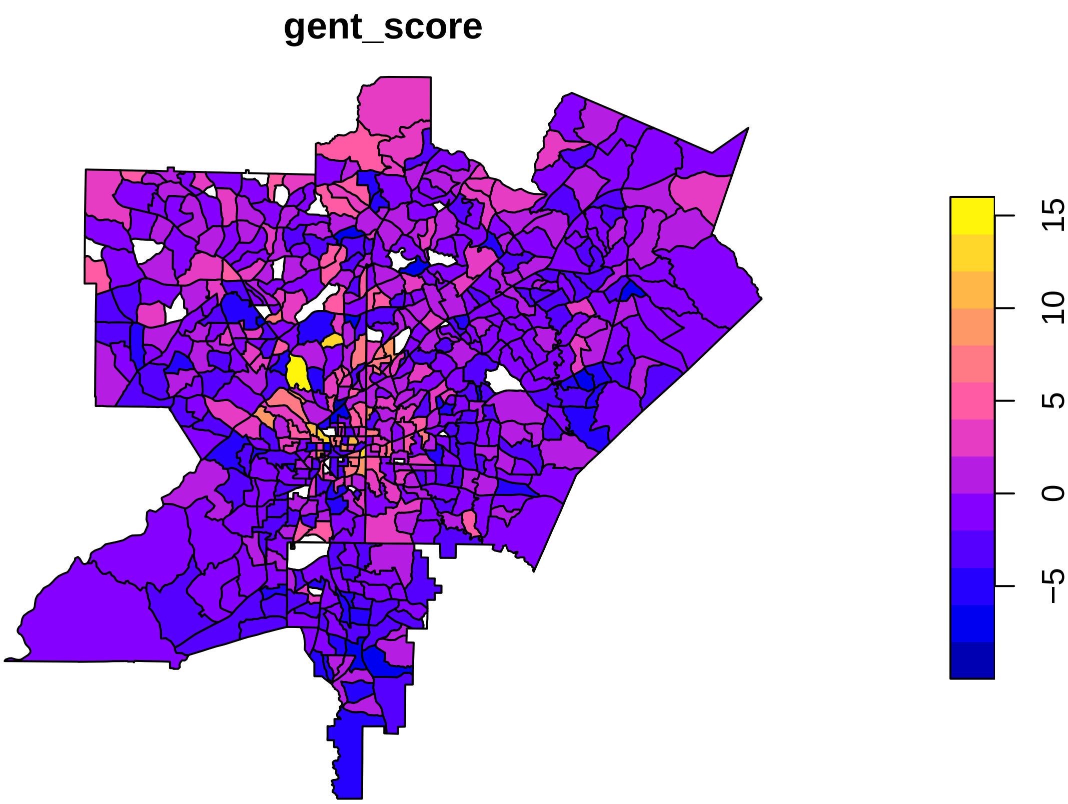
```r
##Merging metro atlanta data with gentrication score data

merge_data <- metro_atlanta_income %>%
  left_join(my.gent.data, by = "GEOID")

head(merge_data)
```

```
## Simple feature collection with 6 features and 6 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -84.45856 ymin: 33.5615 xmax: -84.25199 ymax: 33.80544
## Geodetic CRS:  NAD83
##         GEOID                                         NAME   variable estimate
## 1 13089023501  Census Tract 235.01, DeKalb County, Georgia B19013_001    31649
## 2 13121000200       Census Tract 2, Fulton County, Georgia B19013_001   106045
## 3 13121001500      Census Tract 15, Fulton County, Georgia B19013_001    60356
## 4 13121003100      Census Tract 31, Fulton County, Georgia B19013_001    43170
## 5 13063040203 Census Tract 402.03, Clayton County, Georgia B19013_001    32866
## 6 13063040512 Census Tract 405.12, Clayton County, Georgia B19013_001    37578
##     moe gent_score                       geometry
## 1 10000  1.2797755 MULTIPOLYGON (((-84.28807 3...
## 2 17937  4.1506045 MULTIPOLYGON (((-84.37421 3...
## 3 10036  2.8329505 MULTIPOLYGON (((-84.35987 3...
```

```
## 4  7876 12.1190655 MULTIPOLYGON (((-84.35824 3...
## 5  7081  0.3369793 MULTIPOLYGON (((-84.45853 3...
## 6  7205 -3.3303498 MULTIPOLYGON (((-84.44869 3...
```

Mapping Gentrification Scores

```r
plot(merge_data["gent_score"])
```



Summary

So, as a reminder:

- We read in American Community Survey (ACS) census data from 2016 and mapped income.
- We read in tables of census demographics by tract for 2010 and 2016.
- We calculated standardized differences in each variable.
- We summed up the standarized differences to make a score in each tract.
- We merged our new data to the ACS data.
- We mapped the results.

Notes: Future directions:

- There are some holes in the map and we would still need to checked to make sure we have things in the right place.
- We used 'plot' to make the maps, but there are other, more flexible mapping routines. We've seen ggplot, and tmap has even more. The rest of Chapter 6 at https://walker-data.com/census-r/mapping-census-data-with-r.html has great examples of additional maps!